K.R. Mangalam University - School of Engineering & Technology

Task 2: Unsupervised ML for Anomaly Detection

Dataset: NSL-KDD (KDD Cup 1999 derived) | K-Means, DBSCAN, Isolation Forest, PCA, t-SNE

Name	Yash Raj	Roll No. / Section	2401010008 / D
Programme	B.Tech Computer Science & Engineering	Subject	AI in Cyber Security

1. Objective

Apply unsupervised techniques to find structure and anomalies in network traffic: K-Means with the Elbow Method, DBSCAN with justified parameters, Isolation Forest for outliers, and PCA / t-SNE to visualise the results. Labels are **not** used for training; they are used only afterwards to check how well the unsupervised output matches real attacks.

2. Dataset and Preprocessing

Dataset: NSL-KDD (the cleaned version of the KDD Cup 1999 data) - the KDDTrain+ file with 125,973 records. A random sample of **10,000 records** (random_state = 42; 46.5% attack) was used because DBSCAN and t-SNE are slow on the full file.

- **Features:** 38 numeric features + one-hot protocol_type (tcp / udp / icmp) = 41 features. The high-cardinality columns service (70 values) and flag (11 values) were left out to avoid hundreds of sparse dummy columns that would distort Euclidean distances.
- **Scaling:** StandardScaler (mean 0, variance 1), essential because src_bytes is in millions while rates lie between 0 and 1.

3. Source Code

```
import pandas as pd, numpy as np
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.ensemble import IsolationForest
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score, confusion_matrix

cols = ['duration','protocol_type','service','flag','src_bytes','dst_bytes','land','wrong_fragment','urgent','hot',
 'num_failed_logins','logged_in','num_compromised','root_shell','su_attempted','num_root','num_file_creations',
 'num_shells','num_access_files','num_outbound_cmds','is_host_login','is_guest_login','count','srv_count',
 'serror_rate','srv_serror_rate','rerror_rate','srv_rerror_rate','same_srv_rate','diff_srv_rate','srv_diff_host_rate',
'dst_host_count','dst_host_srv_count','dst_host_same_srv_rate','dst_host_diff_srv_rate','dst_host_same_src_port_rate',
 'dst_host_srv_diff_host_rate','dst_host_serror_rate','dst_host_srv_serror_rate','dst_host_rerror_rate',
 'dst_host_srv_rerror_rate','attack','difficulty']
full = pd.read_csv('KDDTrain+.txt', names=cols).drop(columns='difficulty')
df = full.sample(10000, random_state=42).reset_index(drop=True)      # working sample
df['is_attack'] = (df.attack!='normal').astype(int)
print('Working sample:', df.shape, '| attack share: %.1f%%' % (df.is_attack.mean()*100))
X = pd.get_dummies(df.drop(columns=['attack','is_attack','service','flag']), columns=['protocol_type']).astype(float)
print('Features used:', X.shape[1], '(38 numeric + 3 one-hot protocol; service/flag excluded to limit
    dimensionality)')
Xs = StandardScaler().fit_transform(X)
y = df.is_attack.values

# ---------- K-Means + Elbow
ks=range(1,11); inertia=[]; sil={}
for k in ks:
    km=KMeans(n_clusters=k,random_state=42,n_init=10).fit(Xs); inertia.append(km.inertia_)
    if k>=2: sil[k]=silhouette_score(Xs,km.labels_,sample_size=5000,random_state=42)
print('\nElbow method (k, inertia, silhouette):')
for k,i in zip(ks,inertia): print(f'  k={k:2d}  inertia={i:12.1f}  silhouette={sil.get(k,float("nan")):.3f}')
fig,ax=plt.subplots(1,2,figsize=(10,3.6))
ax[0].plot(list(ks),inertia,'o-',color='#2E5597'); ax[0].set_xlabel('k'); ax[0].set_ylabel('Inertia (WCSS)');
    ax[0].set_title('Elbow method')
ax[1].plot(list(sil),list(sil.values()),'o-',color='#C00000'); ax[1].set_xlabel('k'); ax[1].set_ylabel('Silhouette');
```

```
    ax[1].set_title('Silhouette score')
plt.tight_layout(); plt.savefig('t2_elbow.png',dpi=130); plt.close()
# choose k: elbow by max second difference
red=[(inertia[i-1]-inertia[i])/inertia[i-1]*100 for i in range(1,10)]
print('Relative inertia reduction when moving k-1 -> k (%):', {i+2:round(r,1) for i,r in enumerate(red)})
k_opt=next(k for k in range(2,10) if red[k-1]<10)   # first k whose NEXT step gains <10%
print('Elbow rule (next step improves inertia by <10%%): k = %d'%k_opt)
km=KMeans(n_clusters=k_opt,random_state=42,n_init=10).fit(Xs); df['kmeans']=km.labels_
print('\nK-Means (k=%d) cluster vs attack share:'%k_opt)
print(pd.crosstab(df.kmeans,df.is_attack.map({0:'Normal',1:'Attack'})).to_string())

# ---------- DBSCAN
min_samples=10
nn=NearestNeighbors(n_neighbors=min_samples).fit(Xs); dist=np.sort(nn.kneighbors(Xs)[0][:,-1])
plt.figure(figsize=(6,3.6)); plt.plot(dist,color='#2E5597'); plt.xlabel('Points sorted by distance');
    plt.ylabel(f'{min_samples}-NN distance')
plt.title('k-distance graph for eps selection'); plt.grid(alpha=.3)
print('\nk-distance percentiles (k=%d): 50%%=%.2f 90%%=%.2f 95%%=%.2f
    99%%=%.2f'%(min_samples,*np.percentile(dist,[50,90,95,99])))
res=[]
for eps in [1.0,1.5,2.0,2.5,3.0,4.0]:
    lab=DBSCAN(eps=eps,min_samples=min_samples).fit_predict(Xs)
    nc=len(set(lab))-(1 if -1 in lab else 0); noise=(lab==-1).mean()*100
    res.append((eps,nc,noise)); print(f'  eps={eps:.1f}: clusters={nc:3d}, noise={noise:5.1f}%')

eps=2.0
db=DBSCAN(eps=eps,min_samples=min_samples).fit(Xs); df['dbscan']=db.labels_
nc=len(set(db.labels_))-1; noise=db.labels_==-1
print(f'\nChosen DBSCAN: eps={eps}, min_samples={min_samples} -> {nc} clusters, {noise.sum()} noise points
    ({noise.mean()*100:.1f}%)')
print('Noise points that are real attacks: %d of %d (%.1f%%)'%(y[noise].sum(),noise.sum(),y[noise].mean()*100))
print('Attack rate in clustered points: %.1f%%'%(y[~noise].mean()*100))
plt.savefig('t2_kdist.png',dpi=130); plt.close()

# ---------- Isolation Forest
for cont in [0.05,0.10,0.20]:
    iso=IsolationForest(n_estimators=200,contamination=cont,random_state=42).fit(Xs);
    p=(iso.predict(Xs)==-1).astype(int)
    tn,fp,fn,tp=confusion_matrix(y,p).ravel()
    print(f'IsolationForest contamination={cont:.2f}: flagged={p.sum():5d} precision={tp/(tp+fp):.3f}
    recall={tp/(tp+fn):.3f}')
from sklearn.metrics import roc_auc_score
iso=IsolationForest(n_estimators=200,contamination=0.10,random_state=42).fit(Xs)
df['iso']=(iso.predict(Xs)==-1).astype(int); score=-iso.score_samples(Xs)
print('Chosen contamination=0.10 | ROC-AUC of anomaly score vs attack label: %.3f'%roc_auc_score(y,score))
print('Overlap: DBSCAN-noise & IsolationForest-anomaly =',int((noise & (df.iso==1)).sum()),'records')

# ---------- PCA + t-SNE
pca=PCA(n_components=2,random_state=42); Z=pca.fit_transform(Xs)
print('\nPCA explained variance (PC1, PC2): %.1f%%, %.1f%% (total
    %.1f%%)'%(*(pca.explained_variance_ratio_*100),pca.explained_variance_ratio_.sum()*100))
fig,ax=plt.subplots(2,2,figsize=(10,8))
def sc(a,c,t,cmap='tab10'): a.scatter(Z[:,0],Z[:,1],c=c,s=4,cmap=cmap,alpha=.6); a.set_title(t); a.set_xlabel('PC1');
    a.set_ylabel('PC2')
sc(ax[0,0],y,'True label (0 Normal, 1 Attack)','coolwarm')
sc(ax[0,1],df.kmeans,f'K-Means (k={k_opt})')
sc(ax[1,0],np.where(noise,1,0),'DBSCAN noise (1) vs clustered (0)','coolwarm')
sc(ax[1,1],df.iso,'Isolation Forest anomalies (1)','coolwarm')
plt.tight_layout(); plt.savefig('t2_pca.png',dpi=130); plt.close()
idx=np.random.RandomState(42).choice(len(Xs),3000,replace=False)
T=TSNE(n_components=2,perplexity=30,random_state=42,init='pca').fit_transform(Xs[idx])
fig,ax=plt.subplots(1,2,figsize=(10,4.2))
ax[0].scatter(T[:,0],T[:,1],c=y[idx],s=5,cmap='coolwarm'); ax[0].set_title('t-SNE: true label')
ax[1].scatter(T[:,0],T[:,1],c=df.kmeans.values[idx],s=5,cmap='tab10'); ax[1].set_title(f't-SNE: K-Means clusters
    (k={k_opt})')
plt.tight_layout(); plt.savefig('t2_tsne.png',dpi=130); plt.close()
print('K-Means k=%d purity (majority-class share of each cluster):'%k_opt)
ct=pd.crosstab(df.kmeans,df.is_attack); print((ct.max(axis=1)/ct.sum(axis=1)).round(3).to_string())
print('Overall purity: %.3f'%(ct.max(axis=1).sum()/ct.values.sum()))
```

4. Output

```
Working sample: (10000, 43) | attack share: 46.5%
Features used: 41 (38 numeric + 3 one-hot protocol; service/flag excluded to limit dimensionality)

Elbow method (k, inertia, silhouette):
  k= 1  inertia=    380000.0  silhouette=nan
  k= 2  inertia=    315696.7  silhouette=0.335
  k= 3  inertia=    268773.8  silhouette=0.404
  k= 4  inertia=    234257.7  silhouette=0.407
  k= 5  inertia=    209620.0  silhouette=0.460
  k= 6  inertia=    190181.9  silhouette=0.496
  k= 7  inertia=    172326.0  silhouette=0.510
  k= 8  inertia=    162724.4  silhouette=0.512
```

```
  k= 9  inertia=    152540.8  silhouette=0.526
  k=10  inertia=    144746.1  silhouette=0.525
Relative inertia reduction when moving k-1 -> k (%): {2: 16.9, 3: 14.9, 4: 12.8, 5: 10.5, 6: 9.3, 7: 9.4, 8: 5.6, 9:
    6.3, 10: 5.1}
Elbow rule (next step improves inertia by <10%): k = 5

K-Means (k=5) cluster vs attack share:
is_attack  Attack  Normal
kmeans
0             406    1120
1             946     226
2             519    3995
3               0       3
4            2776       9

k-distance percentiles (k=10): 50%=0.19 90%=1.40 95%=2.07 99%=4.23
  eps=1.0: clusters= 37, noise= 11.5%
  eps=1.5: clusters= 32, noise=  6.2%
  eps=2.0: clusters= 25, noise=  3.2%
  eps=2.5: clusters= 15, noise=  2.1%
  eps=3.0: clusters= 12, noise=  1.6%
  eps=4.0: clusters=  9, noise=  0.8%

Chosen DBSCAN: eps=2.0, min_samples=10 -> 25 clusters, 323 noise points (3.2%)
Noise points that are real attacks: 114 of 323 (35.3%)
Attack rate in clustered points: 46.8%
IsolationForest contamination=0.05: flagged=  500 precision=0.708 recall=0.076
IsolationForest contamination=0.10: flagged= 1000 precision=0.644 recall=0.139
IsolationForest contamination=0.20: flagged= 2000 precision=0.649 recall=0.279
Chosen contamination=0.10 | ROC-AUC of anomaly score vs attack label: 0.703
Overlap: DBSCAN-noise & IsolationForest-anomaly = 151 records

PCA explained variance (PC1, PC2): 18.8%, 12.7% (total 31.5%)
K-Means k=5 purity (majority-class share of each cluster):
kmeans
0    0.734
1    0.807
2    0.885
3    1.000
4    0.997
Overall purity: 0.884
```

4.1 K-Means and the Elbow Method

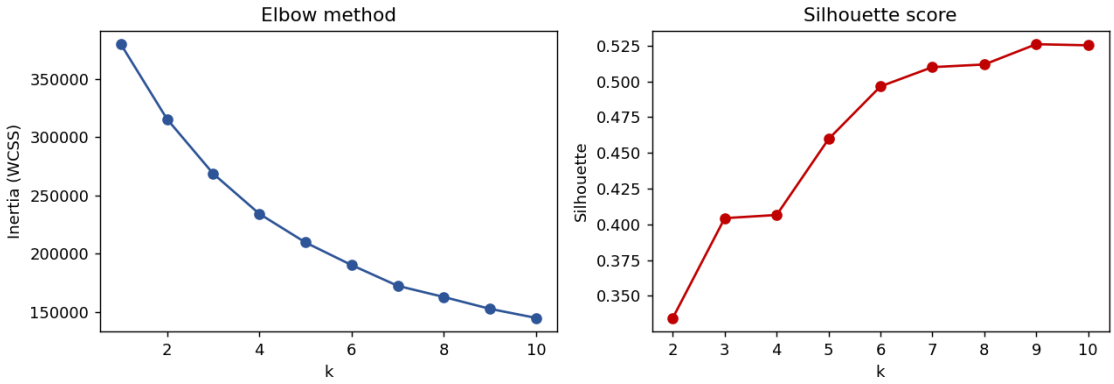


Fig 1: Elbow curve (inertia) and silhouette score for k = 1 to 10.

Choosing k: inertia falls steeply at first and then flattens - the relative improvement from the previous k was 16.9% (k=2), 14.9%, 12.8%, 10.5% (k=5) and 9.3% (k=6). The rule used was "choose the first k whose next step improves inertia by less than 10%", which gives **k = 5**. The silhouette score (0.460 at k=5) keeps rising to 0.526 at k=9 because extra clusters isolate a few tiny groups of extreme points; it was therefore used only as supporting evidence, not as the deciding rule.

K-Means cluster	Attack	Normal	Majority class share
0	406	1,120	73.4% Normal
1	946	226	80.7% Attack
2	519	3,995	88.5% Normal
3	0	3	100% Normal (tiny group)
4	2,776	9	99.7% Attack

Overall cluster purity = 88.4%. Cluster 4 is almost purely attack traffic (mostly dense DoS/SYN-flood-like connections) and cluster 2 is mostly normal traffic; clusters 0 and 1 are mixed, so K-Means separates the two big behaviours but not every attack type.

4.2 DBSCAN

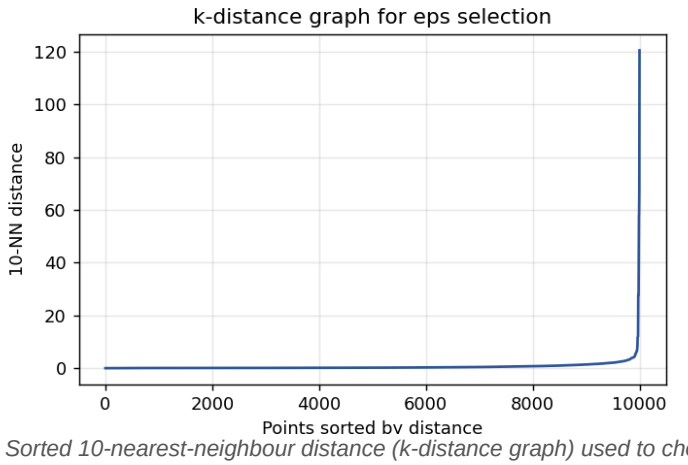


Fig 2: Sorted 10-nearest-neighbour distance (k-distance graph) used to choose eps.

min_samples = 10: a common rule of thumb is min_samples at least ln(n) (about 9 for n = 10,000) and larger values for noisy data; the "2 x dimensions" rule (82 here) would be too strict for 10,000 points and would label most small attack groups as noise. **eps = 2.0:** the k-distance curve is flat (median 0.19) and then bends upward; eps = 2.0 sits at the 95th percentile (2.07) of the 10-NN distances, i.e. at the knee. Trying eps from 1.0 to 4.0 showed that smaller values fragment the data (37 clusters, 11.5% noise at eps = 1.0) and larger values merge everything (9 clusters at eps = 4.0).

Result: **25 clusters and 323 noise points (3.2%)**. Only 35.3% of the noise points (114 of 323) are real attacks, while 46.8% of clustered points are attacks. So DBSCAN noise means "rare / unusual behaviour", which is not the same as "attack": large attack families such as neptune and smurf form dense clusters, while many unusual-but-legitimate connections become noise.

4.3 Isolation Forest

Isolation Forest (200 trees) isolates points with random splits; points that are isolated in few splits get high anomaly scores. The contamination parameter (expected anomaly fraction) was set to 0.10 as a moderate alarm budget:

Contamination	Flagged	Precision (attack)	Recall (attack)
0.05	500	0.708	0.076
0.10 (chosen)	1,000	0.644	0.139
0.20	2,000	0.649	0.279

The anomaly score has a ROC-AUC of 0.703 against the attack label, only modestly better than chance. 151 records were flagged by both DBSCAN (noise) and Isolation Forest, a useful shortlist of unusual connections for an analyst.

4.4 Visualisation with PCA and t-SNE

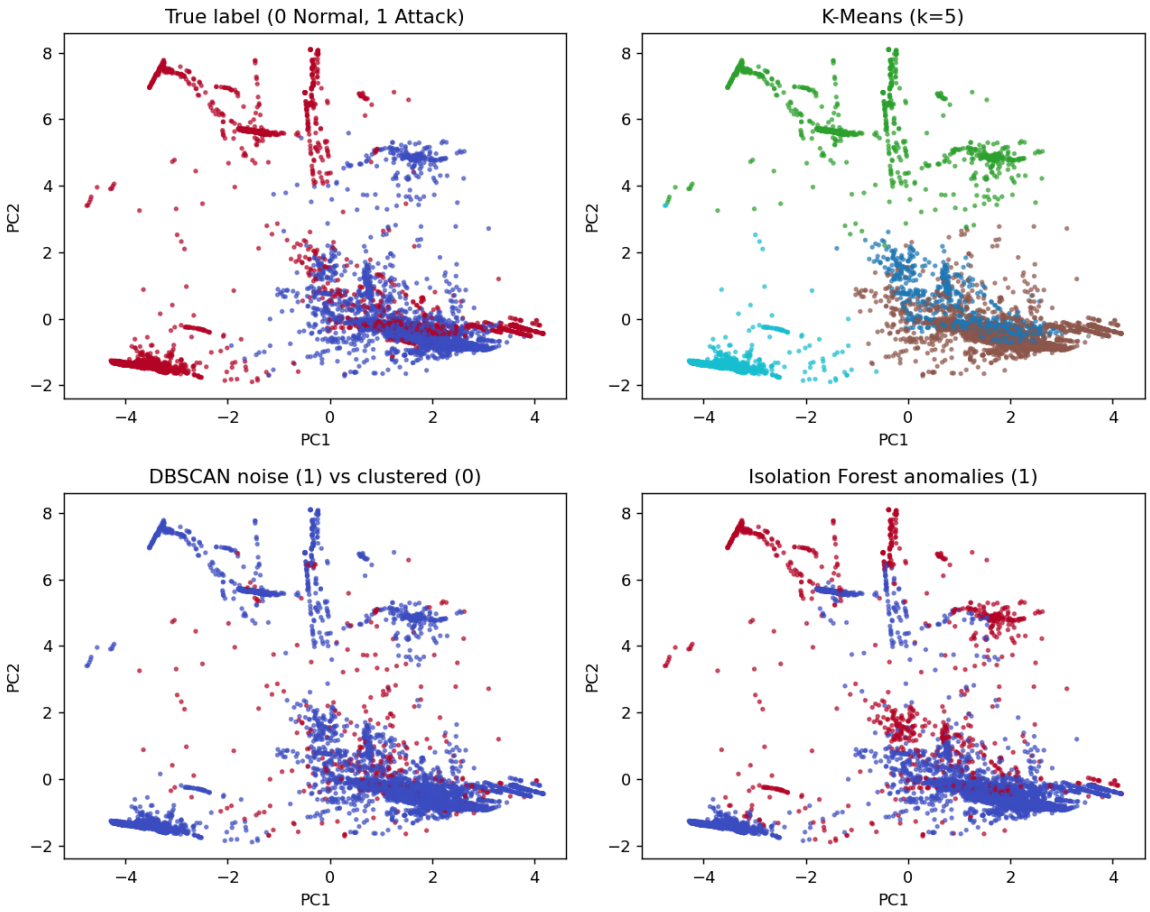


Fig 3: PCA projection coloured by true label, K-Means clusters, DBSCAN noise and Isolation Forest anomalies.

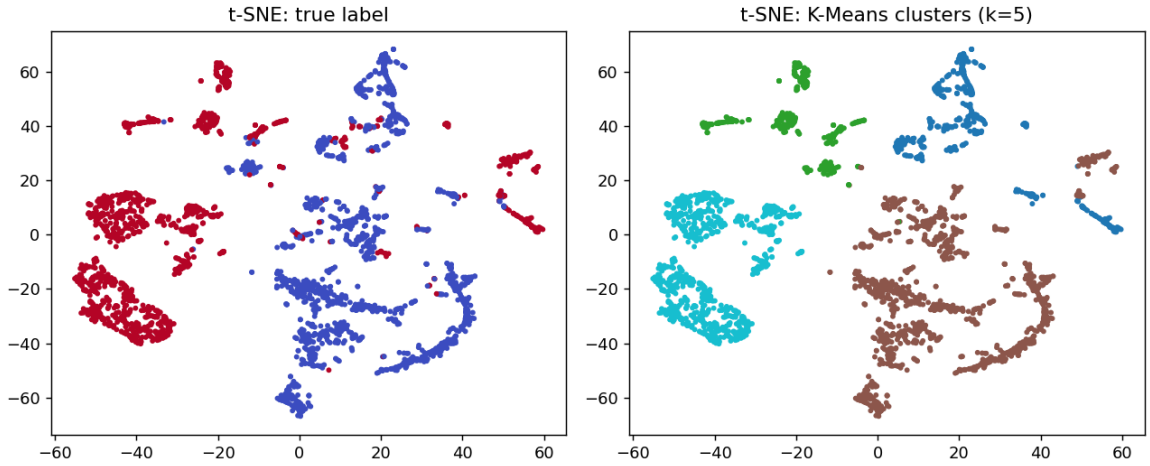


Fig 4: t-SNE (3,000-record sample, perplexity 30) coloured by true label and K-Means cluster.

PCA with two components keeps only 31.5% of the variance (PC1 18.8%, PC2 12.7%), so the 2-D picture is a partial view. t-SNE preserves local neighbourhoods better and shows compact groups that correspond to the K-Means clusters, with attacks and normal traffic mostly in different groups.

5. Explanation of Working

K-Means repeatedly assigns points to the nearest of k centroids and moves each centroid to the mean of its points; the Elbow Method plots the within-cluster sum of squares for k = 1..10 to find the point where adding clusters stops paying off. DBSCAN grows clusters from dense neighbourhoods (at least min_samples points within distance eps) and labels points in sparse regions as noise (-1), so it needs no k and can find irregular shapes. Isolation Forest builds random trees; anomalies are few and different, so

they are separated close to the root.

6. Observations and Limitations

- K-Means (k=5) gave the most useful separation: two clusters are 99%+ pure and overall purity is 88.4% without using any label.
- DBSCAN and Isolation Forest are weaker as attack detectors on this data because attacks are **not rare** (46.5% of the sample); they are designed for rare anomalies. They are better suited to unlabeled real-world logs where attacks are a tiny fraction.
- Results depend on a 10,000-record sample, the chosen features and parameters (k, eps, min_samples, contamination); sensitivity to these is shown in the tables above.
- The dataset is old and synthetic-like (derived from 1998 traffic); the dataset was used for learning only and no real network was monitored.

7. Conclusion

K-Means with k = 5 (Elbow Method) grouped the traffic into meaningful behaviours (88.4% purity), DBSCAN with eps = 2.0 and min_samples = 10 isolated 3.2% of records as unusual, and Isolation Forest flagged the most easily isolated 10% with ROC-AUC 0.70. PCA and t-SNE confirmed the cluster structure visually. Unsupervised methods are good for exploring and shortlisting unusual traffic, but they are less accurate than the supervised models of Task 1 when labelled data is available.